Extreme Value GAMs

GAM.jl Contributors

Table of contents

Introduction

Extreme value theory (EVT) provides a principled framework for modeling the tails of distributions. Two key models are:

- **Generalized Extreme Value (GEV)** distribution for block maxima: $Y \sim \text{GEV}(\mu, \sigma, \xi)$ with CDF

$$F(y) = \exp\left\{-\left[1 + \xi\left(\frac{y-\mu}{\sigma}\right)\right]^{-1/\xi}\right\}$$

- **Generalized Pareto Distribution (GPD)** for threshold exceedances: $Y \mid Y > u \sim \text{GPD}(\sigma, \xi)$ with survival function

$$\bar{F}(y) = \left[1 + \xi\left(\frac{y-u}{\sigma}\right)\right]^{-1/\xi}$$

GAM.jl's `evgam` function fits **multi-parameter GAMs** where each distribution parameter can depend on covariates through smooth functions, following the approach of the R evgam package.

Setup

```
using GAM
using StatsAPI: fitted
using LinearAlgebra: diag
using DataFrames
using Random
using Statistics
```

GEV model

Simulate GEV data

We generate block maxima where the location and scale vary smoothly with a covariate x:

- Location: $\mu(x) = 5 + 2\sin(2\pi x)$
- Log-scale: $\log \sigma(x) = -0.5 + 0.5x$, so $\sigma(x) = \exp(-0.5 + 0.5x)$
- Shape: $\xi = 0.1$ (constant, light upper tail)

```
Random.seed!(42)
n = 500
x = rand(n)

mu_true = 5.0 .+ 2.0 .* sin.(2  .* x)
logsigma_true = -0.5 .+ 0.5 .* x
sigma_true = exp.(logsigma_true)
xi_true = 0.1

function rgev(mu, sigma, xi; rng=Random.default_rng())
    u = rand(rng)
    if abs(xi) < 1e-8
        return mu - sigma * log(-log(u))
    else
        return mu + sigma * ((-log(u))^(-xi) - 1) / xi
    end
end
```

```
y_gev = [rgev(mu_true[i], sigma_true[i], xi_true) for i in 1:n]

df_gev = DataFrame(y=y_gev, x=x)
println("GEV data: n = $(nrow(df_gev)), y range = [$(round(minimum(y_gev), digits=2)), $(rou
```

```
GEV data: n = 500, y range = [1.68, 14.0]
```

Fit the GEV model

We specify one formula per distribution parameter. The GEV has three parameters: location μ, log-scale $\psi = \log \sigma$, and shape ξ.

```
m_gev = evgam(
    [@gam_formula(y ~ s(x, k=10, bs=:cr)),   # location  (x)
     @gam_formula(y ~ s(x, k=8, bs=:cr)),    # log-scale  (x)
     @gam_formula(y ~ 1)],                    # shape   (constant)
    df_gev,
    GEVFamily()
)
```

```
MultiParameterModel{GEVFamily}(GEVFamily(), [4.948349399537339, 1.3266339281181312, 1.909880
0.9366778501147759, -1.9875385368982086, -2.0974960026051575, -1.061992558947692, -
0.1597956759936554, -0.2509970970928611, 0.190958461323388, -0.014360657436445434, 0.2150103
0.23122291689745075, -0.34900997400233197, -0.3823074193346349, -0.22158317404763547, -
0.215345401858029, -0.1686938514265814, -0.21200314546169846, -0.24774148834250473, 0.037822
0.27393707447126797  …  -0.3419854792977815, -0.2431225333885996, -0.24069177688239607, -
0.22105406594837254, -0.37193520645769274, -0.022543639540577153, -0.21668521645068015, -
0.24801896995089684, -0.5105722497949082, -0.31808296925441226], [0.10456912158160805, 0.104
0.09679266246762505 -0.03942170459790328; 1.0 -0.13316235282130828 … -0.11870794851194241 -
0.048087007310564985; … ; 1.0 0.15963508560797618 … -0.2763861875760349 -
0.11221528793412174; 1.0 0.7883037958315805 … -0.15847862147126232 -0.0643266910645724], [1.
0.13324481529434065 -0.05268636693242568; 1.0 -0.3042362533698747 … -0.14354379914332305 -
0.05142077571113935; … ; 1.0 -0.012107852547727649 … -0.35442116128684575 -
0.13794486399898567; 1.0 0.5637770754700124 … -0.24970155935393412 -0.09686741110737508], [1
6.385739399220339, 6.990007468993525], [0.9999999999999998, 0.7672095973125638, 0.9152997892
0.0007800262502307597, 0.06038672766694839, 0.1994080380636328, 0.42176638743587763, 0.19320
5 … 0.00013093028818912675 -0.0004448161347281338; 6.9682097181885e-5 0.014824675896741912 …
0.0007764980895652116 0.0002645490597078857; … ; 0.00013093028818912675 -
0.0007764980895652116 … 0.004988733124466886 -8.533462783811134e-5; -0.0004448161347281338 0
8.533462783811134e-5 0.0011801545073805175], [0.0015728148505199514 6.9682097181885e-
```

```
5 … 0.00013093028818912675 -0.0004448161347281338; 6.9682097181885e-5 0.014824675896741912 …
0.0007764980895652116 0.0002645490597078857; … ; 0.00013093028818912675 -
0.0007764980895652116 … 0.004988733124466886 -8.533462783811134e-5; -0.0004448161347281338 0
8.533462783811134e-5 0.0011801545073805175], 693.6370549108743, 732.0506722944264, [7.391000
```

Examine parameter estimates

```
println("Number of parameters: ", nparams(m_gev))
println("Converged: ", m_gev.converged)
println("Negative log-likelihood: ", round(m_gev.nll, digits=2))
println("REML score: ", round(m_gev.reml, digits=2))
```

```
Number of parameters: 3
Converged: true
Negative log-likelihood: 693.64
REML score: 732.05
```

Location parameter coefficients and fitted values:

```
mu_coefs = param_coef(m_gev, 1)
mu_hat = param_eta(m_gev, 1)
println("Location coefficients (first 5): ", round.(mu_coefs[1:min(5, length(mu_coefs))], dig
println("Location fitted range: [$(round(minimum(mu_hat), digits=2)), $(round(maximum(mu_hat)
println("Location true range: [$(round(minimum(mu_true), digits=2)), $(round(maximum(mu_true)
```

```
Location coefficients (first 5): [4.948, 1.327, 1.91, 1.83, 0.568]
Location fitted range: [2.77, 6.98]
Location true range: [3.0, 7.0]
```

Log-scale parameter:

```
psi_coefs = param_coef(m_gev, 2)
psi_hat = param_eta(m_gev, 2)
println("Log-scale coefficients (first 5): ", round.(psi_coefs[1:min(5, length(psi_coefs))],
println("Log-scale fitted range: [$(round(minimum(psi_hat), digits=2)), $(round(maximum(psi_h
println("Log-scale true range: [$(round(minimum(logsigma_true), digits=2)), $(round(maximum(l
```

```
Log-scale coefficients (first 5): [-0.251, 0.191, -0.014, 0.215, 0.109]
Log-scale fitted range: [-0.65, 0.11]
Log-scale true range: [-0.5, -0.0]
```

Shape parameter (constant):

```
xi_coefs = param_coef(m_gev, 3)
println("Shape coefficient: ", round(xi_coefs[1], digits=4))
println("True shape: ", xi_true)
```

```
Shape coefficient: 0.1046
True shape: 0.1
```

Compare fitted vs true functions

```
ord = sortperm(x)
x_sorted = x[ord]

cor_mu = cor(mu_hat, mu_true)
cor_psi = cor(psi_hat, logsigma_true)
println("Correlation (fitted vs true location): ", round(cor_mu, digits=4))
println("Correlation (fitted vs true log-scale): ", round(cor_psi, digits=4))
println("RMSE (location): ", round(sqrt(mean((mu_hat .- mu_true).^2)), digits=3))
println("RMSE (log-scale): ", round(sqrt(mean((psi_hat .- logsigma_true).^2)), digits=3))
```

```
Correlation (fitted vs true location): 0.9981
Correlation (fitted vs true log-scale): 0.6607
RMSE (location): 0.107
RMSE (log-scale): 0.114
```

GPD model

Simulate GPD data

For threshold exceedances, we simulate GPD data with covariate-dependent scale:

- Log-scale: $\log \sigma(x) = 0.5 \sin(2\pi x)$
- Shape: $\xi = 0.15$ (constant)

```
Random.seed!(42)
n_gpd = 500
x_gpd = rand(n_gpd)

logsigma_gpd_true = 0.5 .* sin.(2  .* x_gpd)
sigma_gpd_true = exp.(logsigma_gpd_true)
xi_gpd_true = 0.15

function rgpd(sigma, xi; rng=Random.default_rng())
    u = rand(rng)
    if abs(xi) < 1e-8
        return -sigma * log(u)
    else
        return sigma * (u^(-xi) - 1) / xi
    end
end

y_gpd = [rgpd(sigma_gpd_true[i], xi_gpd_true) for i in 1:n_gpd]

df_gpd = DataFrame(y=y_gpd, x=x_gpd)
println("GPD data: n = $(nrow(df_gpd)), y range = [$(round(minimum(y_gpd), digits=2)), $(rou
```

```
GPD data: n = 500, y range = [0.0, 25.09]
```

Fit the GPD model

The GPD has two parameters: log-scale $\psi = \log \sigma$ and shape ξ.

```
m_gpd = evgam(
    [@gam_formula(y ~ s(x, k=10, bs=:cr)),   # log-scale  (x)
     @gam_formula(y ~ 1)],                    # shape   (constant)
    df_gpd,
    GPDFamily()
)
```

```
MultiParameterModel{GPDFamily}(GPDFamily(0.0), [-0.03715169266402914, 0.2565637058532435, 0.4
0.2729500664875058, -0.32160569561565844, -0.24466911329045732, -0.15155601882693026, -
0.022290943784345006, 0.1825253154353981], [[0.3486486766398819, 0.3375768688735314, 0.40354
0.20301225643103504, -0.08478511929977922, 0.18189828043697784, -0.30187427265893313, -
0.0970930120378658, 0.1751832626127269  …  0.3304403336785182, -0.3068733710221351, 0.212131
```

```
0.12317394385284705, -0.36414760387635475, 0.22956794235638556, -0.001913535848340258, 0.1353
0.09679266246762505 -0.03942170459790328; 1.0 -0.13316235282130828 … -0.11870794851194241 -
0.048087007310564985; … ; 1.0 0.15963508560797618 … -0.2763861875760349 -
0.11221528793412174; 1.0 0.7883037958315805 … -0.15847862147126232 -0.0643266910645724], [1.0
3.984853993532385], [1.0000000000000004, 0.2285445432902115, 0.4832183638497569, 0.545067759
5 -0.002178941904307682; 0.0004407316298123316 0.008794594625956002 … -
0.0011575755378985845 -0.0004856933926868101; … ; 6.773210405381969e-5 -
0.0011575755378985845 … 0.035892801959681586 -0.0001537245463555683; -0.002178941904307682 -
0.0004856933926868101 … -0.0001537245463555683 0.0026230763791754495], [0.004534890198816263
5 -0.002178941904307682; 0.0004407316298123316 0.008794594625956002 … -
0.0011575755378985845 -0.0004856933926868101; … ; 6.773210405381969e-5 -
0.0011575755378985845 … 0.035892801959681586 -0.0001537245463555683; -0.002178941904307682 -
0.0004856933926868101 … -0.0001537245463555683 0.0026230763791754495], 572.6868123730114, 589
```

Examine GPD estimates

```
println("Converged: ", m_gpd.converged)
println("Negative log-likelihood: ", round(m_gpd.nll, digits=2))

psi_gpd_hat = param_eta(m_gpd, 1)
xi_gpd_hat = param_coef(m_gpd, 2)

println("Log-scale fitted range: [$(round(minimum(psi_gpd_hat), digits=2)), $(round(maximum(p
println("Log-scale true range: [$(round(minimum(logsigma_gpd_true), digits=2)), $(round(maxi
println("Shape estimate: ", round(xi_gpd_hat[1], digits=4))
println("True shape: ", xi_gpd_true)

cor_gpd = cor(psi_gpd_hat, logsigma_gpd_true)
println("Correlation (fitted vs true log-scale): ", round(cor_gpd, digits=4))
```

```
Converged: true
Negative log-likelihood: 572.69
Log-scale fitted range: [-0.38, 0.41]
Log-scale true range: [-0.5, 0.5]
Shape estimate: 0.1825
True shape: 0.15
Correlation (fitted vs true log-scale): 0.9784
```

Model structure

The `MultiParameterModel` returned by `evgam` stores:

```
println("Type: ", typeof(m_gev))
println("Total coefficients: ", length(m_gev.coefficients))
println("EDFs: ", round.(m_gev.edf, digits=2))
println("Log smoothing parameters: ", round.(m_gev.sp, digits=2))
println("Covariance matrix size: ", size(m_gev.Vp))
```

```
Type: MultiParameterModel{GEVFamily}
Total coefficients: 19
EDFs: [1.0, 0.77, 0.92, 0.85, 0.85, 0.86, 0.84, 0.84, 0.77, 0.78, 1.0, 0.1, 0.03, -
0.0, 0.06, 0.2, 0.42, 0.19, 1.0]
Log smoothing parameters: [-6.39, 6.99]
Covariance matrix size: (19, 19)
```

Standard errors via the posterior covariance:

```
se_all = sqrt.(diag(m_gev.Vp))
println("Standard errors (first 5): ", round.(se_all[1:min(5, length(se_all))], digits=4))
```

```
Standard errors (first 5): [0.0397, 0.1218, 0.0948, 0.1044, 0.1147]
```

Summary

GAM.jl's `evgam` provides:

Feature	Description
`GEVFamily()`	GEV distribution (3 parameters: , log ,)
`GPDFamily()`	GPD distribution (2 parameters: log ,)
Multi-formula interface	One formula per distribution parameter
REML smoothing	Automatic smoothing parameter estimation
`param_coef(m, k)`	Coefficients for parameter k
`param_eta(m, k)`	Fitted linear predictor for parameter k